



DEPARTAMENTO
DE COMPUTACION

Facultad de Ciencias Exactas y Naturales - UBA

Aprendizaje Automático

Clase 4:

Clasificadores: Modelos Discriminativos y Generativos

$$P(A|B) = \frac{P(B|A)P(A)}{P(B)}$$

Clasificadores: Modelos Discriminativos y Generativos

El clasificador óptimo de Bayes

Al construir un clasificador, en general estamos interesados en disminuir el error en “la realidad”. Es decir, minimizar

$$Err_{true}(h) = \mathbb{E}_x[error(h(x), y)]$$

En clasificación, este error puede definirse como **1 if $h(x) \neq y$ else 0** (se lo llama “riesgo”). Se puede demostrar que el clasificador que minimiza el riesgo es:

El **clasificador óptimo de Bayes**, un modelo probabilístico que devuelve la predicción más probable para un nuevo ejemplo, dado los valores de sus atributos. Es decir,

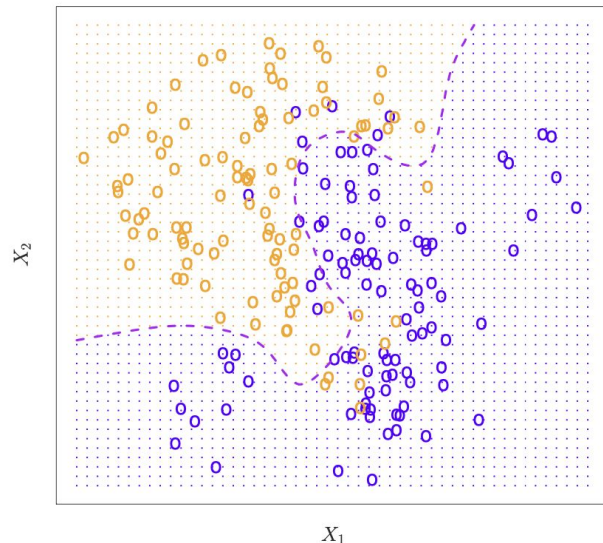
$$Pred(x^{(i)}) = \arg \max_{c \in Clases} P(Y = c | X = x^{(i)})$$

En teoría, siempre nos gustaría predecir respuestas cualitativas utilizando el clasificador de Bayes. ¿Por qué no hacerlo entonces?

✗ Para datos reales, no conocemos la distribución condicional de Y dada X, por lo que calcular el clasificador de Bayes es **imposible**.

Por lo tanto, el clasificador de Bayes sirve como un estándar inalcanzable. En casos específicos (datos simulados), permite comparar diferentes métodos.

Fronteras de decisión según un clasificador de tipo “Bayes Optimal Classifier” para datos simulados .



(notar que la frontera hubiera sido la misma sin importar que muestra se visualice aquí)

Modelos Discriminativos

Enfoques (parte 1)

$$P(Y = c|X = x^{(i)}) \quad \forall c \in Clases$$

Muchos enfoques intentan **estimar** esta probabilidad a partir de un conjunto de **datos de entrenamiento**. Es decir, modelar (aprender):

$$\hat{P}(Y=c|X=x^{(i)}) \quad \forall c \in Clases$$

Luego clasificar una observación dada en la clase con la probabilidad estimada más alta:

$$Pred(x^{(i)}) = \arg \max_{c \in Clases} \hat{P}(Y=c|X=x^{(i)})$$

Estos se conocen como modelos **DISCRIMINATIVOS**

(más adelante en la clase veremos que no es el único enfoque).

(algunos) ejemplos de modelos discriminativos:

- **Árboles de decisión** (ya lo vimos)
- **K-vecinos más cercanos** (next)
- **Support Vector Machines (SVM)** (next)
- **Regresión logística** (proximamente)
- **Random Forest (y otros ensambles)** (proximamente)
- **Maximum-entropy Markov models**
- **Conditional random fields**
- **Redes neuronales (no todas)** (proximamente)

modelos discriminativos

K Vecinos Más Cercanos

KNN

NOTHING MAKES YOU MORE
TOLERANT OF A NOISY
NEIGHBOR'S PARTY THAN
BEING THERE.

— FRANKLIN P. JONES

$$Pred(x^{(i)}) = \arg \max_{c \in Clases} \hat{P}(Y=c|X=x^{(i)})$$

K vecinos más cercanos (KNN)

Método discriminativo

Estima la probabilidad condicional para la clase c dado $\mathbf{x}$ como: La fracción de puntos en $\mathbf{N}_{D,k}(\mathbf{x})$.

$\mathbf{N}_{D,k}(\mathbf{x})$ = los k vecinos más cercanos a $\mathbf{x}$ en el conjunto $\mathbf{D}$, cuyas etiquetas sean iguales a c .

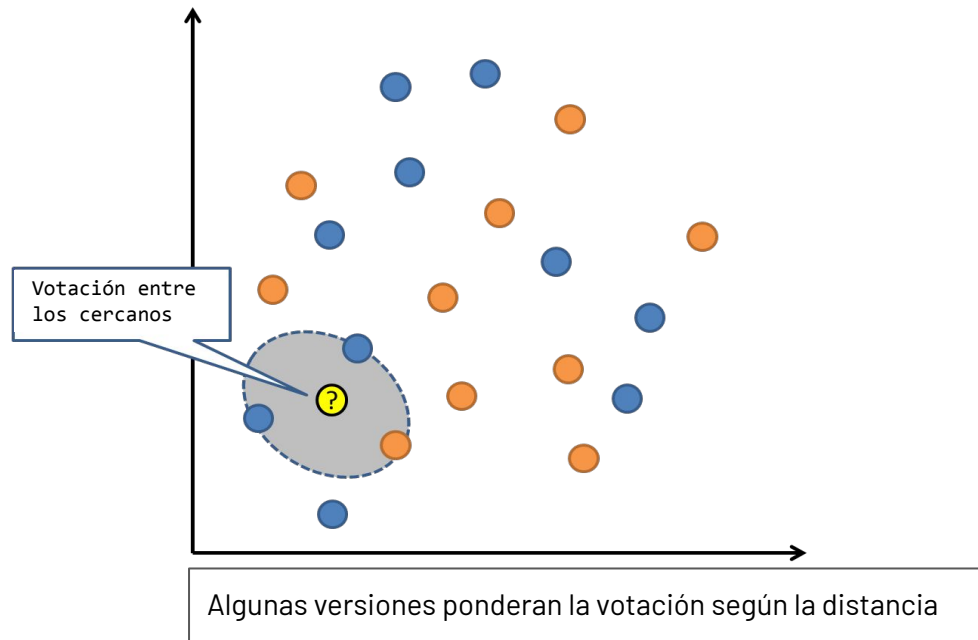
$$Pred(x^{(i)}) = \arg \max_{c \in Clases} P(Y = c|X = x^{(i)})$$

$$\approx \arg \max_{c \in Clases} \sum_{i \in \mathcal{N}_{D,k}(x)} I(y^{(i)} = c)$$

+ sesgo inductivo
+ datos

$$Pred_{proba}(x^{(i)}, c) = P(Y = c|X = x^{(i)})$$

$$\approx \frac{1}{k} \sum_{i \in \mathcal{N}_{D,k}(x)} I(y^{(i)} = c)$$



$$Pred(x^{(i)}) = \arg \max_{c \in Clases} \hat{P}(Y=c|X=x^{(i)})$$

K vecinos más cercanos (KNN)

Método discriminativo

¿Cómo se implementa? Algoritmo de asignación a la instancia $x^{(i)}$:

1. Computar la distancia $D(\mathbf{x}^{(i)}, \mathbf{x}^{(j)})$ para todo punto de entrenamiento $\mathbf{x}^{(j)}$.
2. Seleccionar las k instancias más cercanas y sus etiquetas.
3. Devolver la etiqueta más frecuente (o la proporción si es proba).

Pregunta (clásico de entrevistas)

Quiero predecir si una casa tendrá techo verde o no.

¿Qué pasa si los atributos son los siguientes y aplico KNN?

X_1 : distancia al obelisco (*medido en metros*),

X_2 : cantidad de personas que viven en la cuadra (*de 1 a 500*).

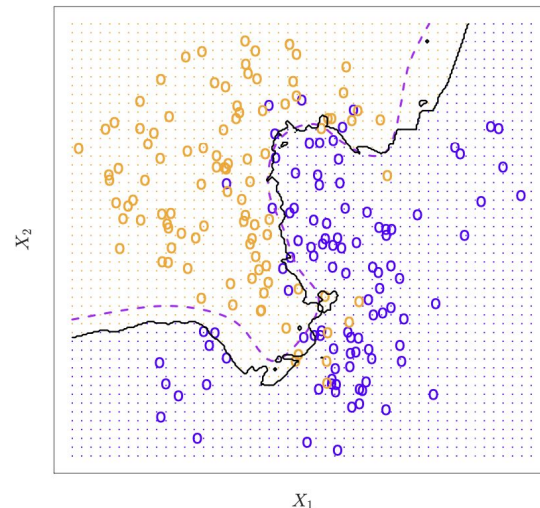
X_3 : ¿tiene jardín?: *si, no (1 o 0)*.

¿Qué distancia uso? ¿Euclidiana? ¿Hago algo antes? ¿Por qué no pasaba en árboles?

Fronteras de decisión según un clasificador de tipo **KNN** con **K=10**.

(¿cómo dibujarían esto?)

Línea punteada: **Bayes Optimal Classifier**



¿Cómo cambian las fronteras de decisión si cambio el K?

$$Pred(x^{(i)}) = \arg \max_{c \in Clases} \hat{P}(Y=c|X=x^{(i)})$$

K vecinos más cercanos (KNN)

Método discriminativo

¿Cómo se implementa? Algoritmo de asignación a la instancia $\mathbf{x}^{(i)}$:

1. Computar la distancia $D(\mathbf{x}^{(i)}, \mathbf{x}^{(j)})$ para todo punto de entrenamiento $\mathbf{x}^{(j)}$.
2. Seleccionar las k instancias más cercanas y sus etiquetas.
3. Devolver la etiqueta más frecuente (o la proporción si es proba).

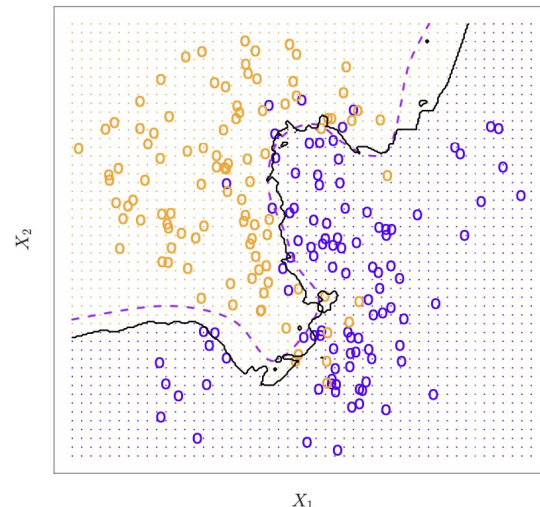
- ¿Cuánto ocupa en memoria una vez entrenado el modelo ?
- ¿Modelo?
- ¿Cuál es el algoritmo de entrenamiento?
- ¿Entrenamiento?

Disclaimer: Hasta acá llega nuestra explicación. Hay muchísimo más para explorar sobre este algoritmo, como por ejemplo: diferentes métricas de distancia, elección óptima de K , búsqueda eficiente de vecinos (KD-Trees, Ball Trees, Locality Sensitive Hashing), complejidad temporal y espacial de KNN, escalabilidad en grandes volúmenes de datos, variantes probabilísticas y múltiples aplicaciones prácticas, entre otros.

Fronteras de decisión según un clasificador de tipo **KNN** con **K=10**.

(¿cómo dibujarían esto?)

Línea punteada: **Bayes Optimal Classifier**



modelos discriminativos

Support Vector Machines

SVM

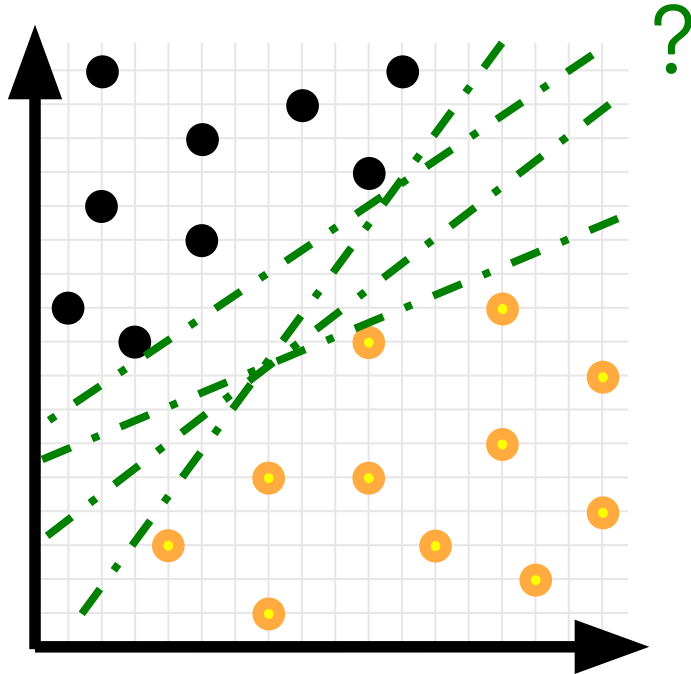
Recomendado [StatQuest - SVM \(youtube\)](#)



$$Pred(x^{(i)}) = \arg \max_{c \in Clases} \hat{P}(Y=c|X=x^{(i)})$$

Support Vector Machine (SVM)

Margen



Idea: buscar una recta que separe las clases lo mejor posible, sin tener que modelar la distribución de los datos de cada clase.

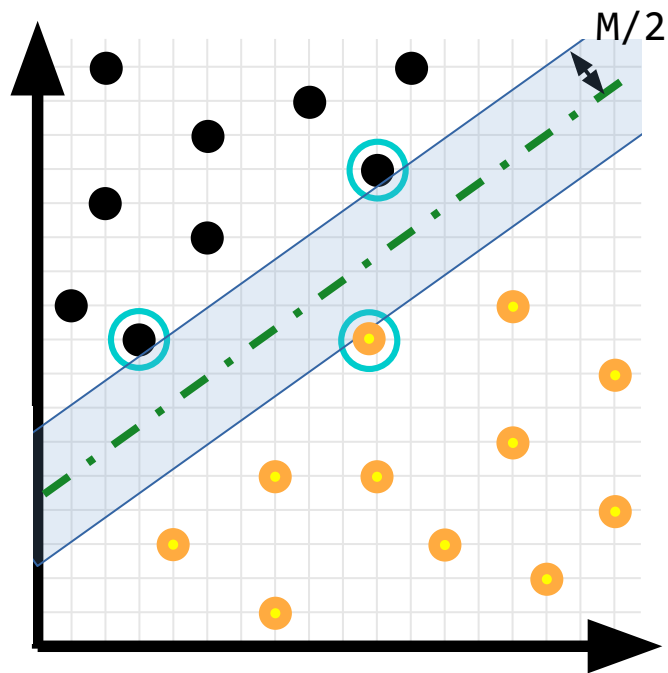
¿Cómo podemos elegir esa recta?
(supongamos por un rato que existe dicha recta)

Buscaremos el **hiperplano de margen máximo** (también conocido como hiperplano de separación óptimo), que es el hiperplano de separación que está más alejado de las observaciones de entrenamiento.

$$Pred(x^{(i)}) = \arg \max_{c \in Clases} \hat{P}(Y=c|X=x^{(i)})$$

Support Vector Machine (SVM)

Margen



Hiperplano de margen máximo (recta verde en este caso). El hiperplano de dimensión **(d-1)** tal que la distancia a las instancias más cercanas es máxima y todas las instancias quedan del lado correcto del plano. Recordar que las instancias viven en el espacio de atributos de dim **d**.

Margen M:

Distancia de las instancias más cercanas a la recta (hiperplano) de decisión (x2).

“Support Vectors” (vectores de soporte, en este caso con círculos celestes): Instancias más cercanas al hiperplano de decisión

El algoritmo buscará maximizar M.

- **Problema de optimización** (mundo **prog. cuadrática**).
Pinta: “Maximizar el margen con tal y tal restricción”.
Existe una solución eficiente. Ahora lo veremos más en detalle.

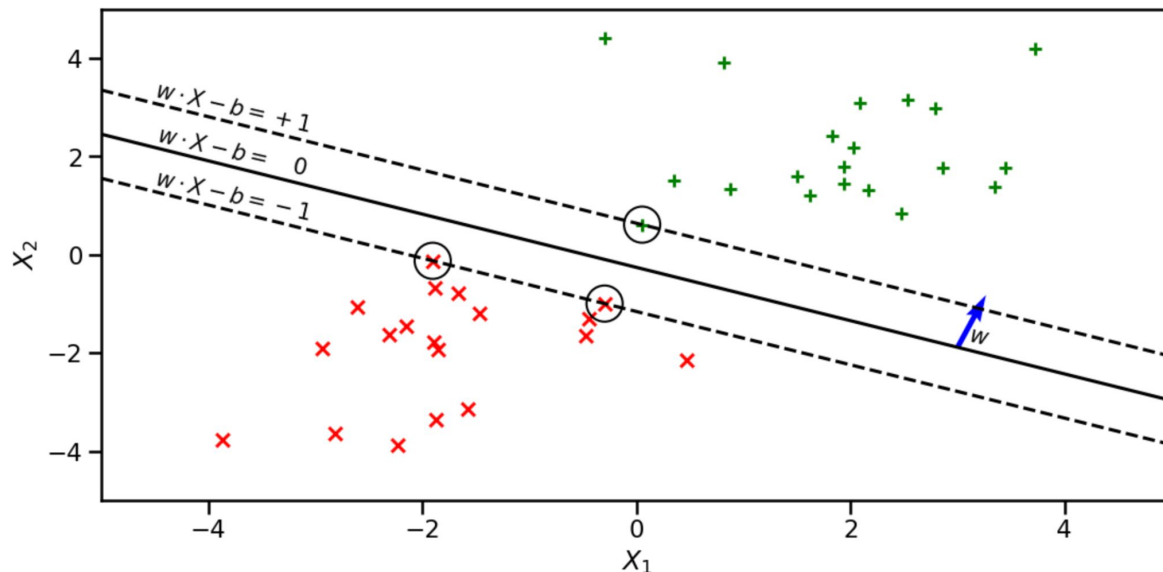
$$Pred(x^{(i)}) = \arg \max_{c \in Clases} \hat{P}(Y=c|X=x^{(i)})$$

Support Vector Machine (SVM)

Definiciones

Consideremos etiquetas +1 y -1 para las clases positiva y negativa, respectivamente. Recordemos la ecuación de un hiperplano: $\mathbf{w}^T \mathbf{X} - \mathbf{b} = 0$. Esto hará que los vectores de soporte cumplan con $\mathbf{w}^T \mathbf{x}^{(i)} - \mathbf{b} = 1$ y el resto $(\mathbf{w}^T \mathbf{x}^{(i)} - \mathbf{b}) > 1$ (o equiv. para negativos)

Notar que $\mathbf{w}$ es un vector normal a la frontera de decisión



El tamaño del margen es $2/\|\mathbf{w}\|_2$ (demo al final de las slides, para los que quieran)

$$Pred(x^{(i)}) = \arg \max_{c \in Clases} \hat{P}(Y=c|X=x^{(i)})$$

Support Vector Machine (SVM)

Maximización del Margen

El tamaño del margen es $2/\|\mathbf{w}\|$: Queremos maximizarlo. O equivalentemente **min** $\|\mathbf{w}\|$

Pero con dos restricciones:

- Ningún punto positivo cae del lado incorrecto: $\mathbf{w}^\top \mathbf{x}_{(+)} - b \geq 1$ para todo $\mathbf{x}_{(+)}$
- Ningún punto negativo cae del lado incorrecto: $\mathbf{w}^\top \mathbf{x}_{(-)} - b \leq -1$ para todo $\mathbf{x}_{(-)}$

Escrito de otra forma: $\min_{\mathbf{w}, b} \|\mathbf{w}\|_2 \quad \text{tal que} \quad y^{(i)}(\mathbf{w}^\top \mathbf{x}^{(i)} - b) \geq 1 \quad \forall (\mathbf{x}^{(i)}, \mathbf{y}^{(i)}) \in D_{train}$

A saber, $\mathbf{y}^{(i)}(\mathbf{w}^\top \mathbf{x}^{(i)} - b) = 1$ son los vectores de soporte

La solución a este problema se puede encontrar utilizando técnicas de **optimización convexa**, (formulación dual de Lagrange). No lo veremos, pero tiene esta pinta:

Solución (en donde α_i y b se aprenden a partir de los datos) $f(\mathbf{x}^{(k)}) = \text{sign} \left(\sum_{i=1}^n \alpha_i \mathbf{y}^{(i)} \mathbf{x}^{(i)\top} \mathbf{x}^{(k)} + b \right)$

Support Vector Machine (SVM)

Separaciones no lineales - Atributos Slack

$$Pred(x^{(i)}) = \arg \max_{c \in Clases} \hat{P}(Y=c|X=x^{(i)})$$

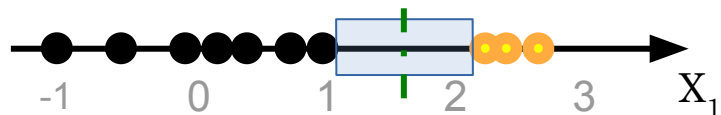
Notación

(instancia / atributos)

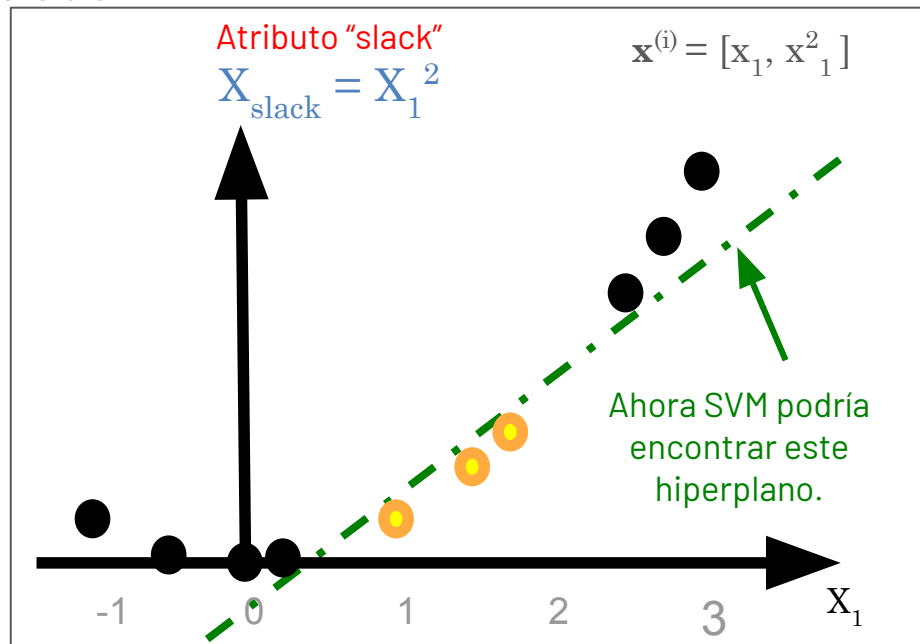
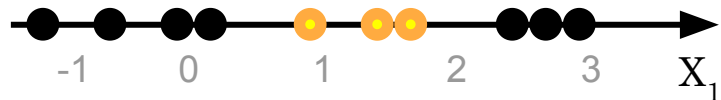
$$\mathbf{x}^{(i)} = [x_1, x_2, \dots, x_p]$$

Supongamos para estos ejemplos $\mathbf{x}^{(i)} = [x_1]$

En este caso, es sencillo encontrar un punto que separe bien



¿Qué sucede en un caso como este? Es un dataset linealmente separable?

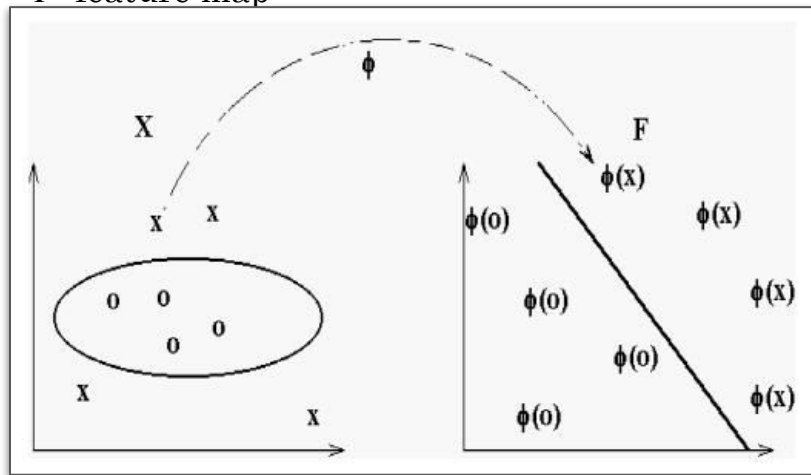


Support Vector Machine (SVM)

Separaciones no lineales - Atributos Slack

Transformación de vectores de atributos.

Φ “feature map”



Ejemplo de una transformación polinómica de grado 2:

$$\Phi_{poli}(\mathbf{x}) = \Phi_{poli}([x_1, x_2]) = (x_1, x_2, x_1x_2, x_1^2, x_2^2)$$

$$\dim(\Phi_{poli}(\mathbf{x})) \in O(p^d) \quad (p \text{ dim } \mathbf{x}, d \text{ grado polinomio})$$

$$Pred(x^{(i)}) = \arg \max_{c \in Clases} \hat{P}(Y=c|X=x^{(i)})$$

$$\min \|\mathbf{w}\|$$

Ya mencionamos que (sin feature maps), la solución tiene la siguiente forma:

$$f(\mathbf{x}^{(k)}) = \text{sign} \left(\sum_{i=1}^n \alpha_i \mathbf{y}^{(i)} \mathbf{x}^{(i)\top} \mathbf{x}^{(k)} + b \right)$$

Al utilizar feature maps, la solución tiene la pinta:

$$f(\mathbf{x}^{(k)}) = \text{sign} \left(\sum_{i=1}^n \alpha_i \mathbf{y}^{(i)} \Phi(\mathbf{x}^{(i)})^\top \Phi(\mathbf{x}^{(k)}) + b \right)$$

Expandir las transformaciones $\Phi(\cdot)$ es muy costoso (o a veces imposible).

Clave: la solución **sólo usa** “ $\Phi(\mathbf{x}^{(i)})^\top \Phi(\mathbf{x}^{(k)})$ ” (el producto interno, $\langle \Phi(\mathbf{x}^{(i)}), \Phi(\mathbf{x}^{(k)}) \rangle$) donde $\mathbf{x}^{(i)}$, $\mathbf{x}^{(k)}$ son dos instancias distintas.

¿Podremos calcular $\langle \Phi(\mathbf{x}^{(i)}), \Phi(\mathbf{x}^{(k)}) \rangle$ eficientemente?

$$Pred(x^{(i)}) = \arg \max_{c \in Clases} \hat{P}(Y=c|X=x^{(i)})$$

Support Vector Machine (SVM)

Separaciones no lineales - Kernel Trick

Kernel: Generalización del producto interno que nos permite operar con nuevos atributos en forma implícita.

$\mathbf{K}(\mathbf{x}^{(1)}, \mathbf{x}^{(2)}) = \langle \Phi(\mathbf{x}^{(1)}), \Phi(\mathbf{x}^{(2)}) \rangle$ donde $\mathbf{x}^{(1)}, \mathbf{x}^{(2)}$ son dos instancias.

Si un algoritmo (ej. SVM) puede expresarse en términos de productos internos entre instancias, reemplazamos las apariciones de $\langle \Phi(\mathbf{x}^{(1)}), \Phi(\mathbf{x}^{(2)}) \rangle$ por $\mathbf{K}(\mathbf{x}^{(1)}, \mathbf{x}^{(2)})$. Sin nunca tener que computar $\Phi(\mathbf{x})$.

Así, ejecutamos SVM **implícitamente** en dimensiones superiores (nunca computamos $\Phi(\cdot)$)

Sugerencia,

- ver: <https://www.youtube.com/watch?v=Q7vT0-5VII>
- **0 para mucho más detalle:**
<https://www.youtube.com/watch?v=8NYo0iRANpg>

Ejemplos

Kernel Lineal

$$K(\mathbf{x}^{(i)}, \mathbf{x}^{(j)}) = \mathbf{x}^{(i)\top} \cdot \mathbf{x}^{(j)}$$

Kernel Polinómico (dim d)

$$K(\mathbf{x}^{(i)}, \mathbf{x}^{(j)}) = (\mathbf{x}^{(i)\top} \cdot \mathbf{x}^{(j)} + c)^d \quad \dim(\Phi(\mathbf{x})) \in O(\mathbb{R}^{(p^d)})$$

Gaussian Kernel

$$K(\mathbf{x}^{(i)}, \mathbf{x}^{(j)}) = e^{(-\gamma \|\mathbf{x}^{(i)} - \mathbf{x}^{(j)}\|^2)} \quad \Phi(\mathbf{x}) \in \mathbb{R}^\infty$$

Kernel Sigmoido

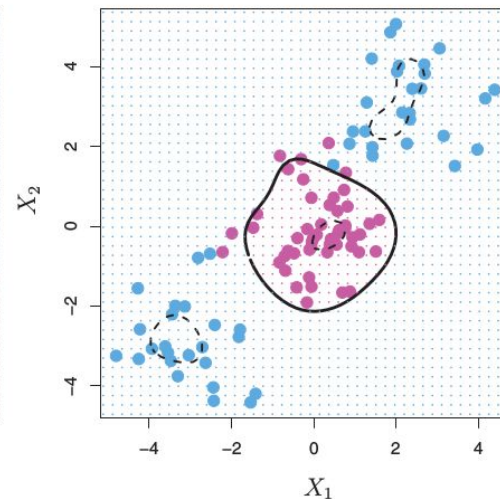
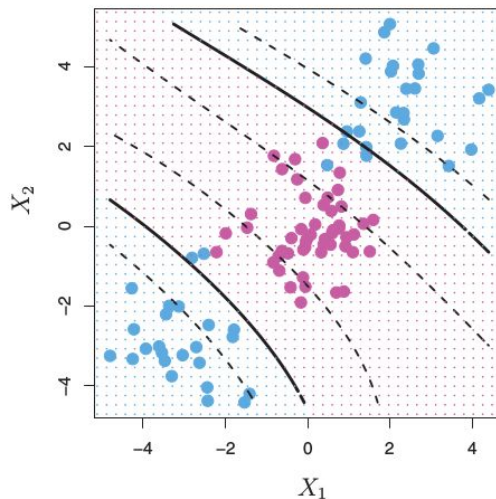
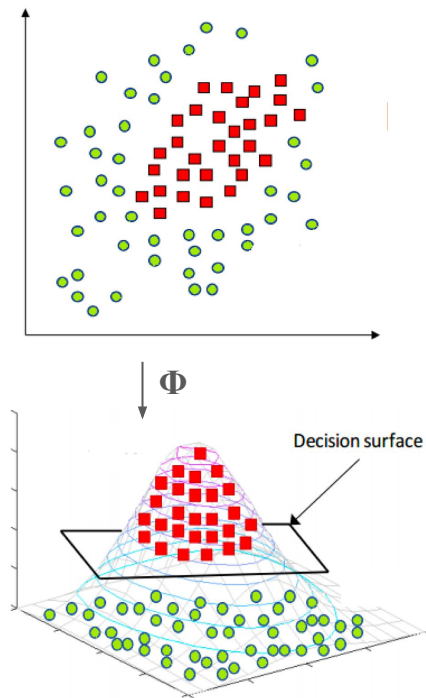
$$K(\mathbf{x}^{(i)}, \mathbf{x}^{(j)}) = \tanh(\gamma(\mathbf{x}^{(i)\top} \cdot \mathbf{x}^{(j)}) + r)$$

scikit-learn.org/stable/modules/svm.html#svm-kernels

$$Pred(x^{(i)}) = \arg \max_{c \in Clases} \hat{P}(Y=c|X=x^{(i)})$$

Support Vector Machine (SVM)

Separaciones no lineales - Kernel Trick



Izquierda: Se aplica SVM con un kernel polinómico de grado 3.

Derecha: Se aplica SVM con un kernel radial.

En este ejemplo, ambos kernels son capaces de capturar la frontera de decisión. [ISLR, cap9, FIGURA 9.9]

[Rizwan, A., Iqbal, N., Ahmad, R., & Kim, D. H. (2021). WR-SVM model based on the margin radius approach for solving the minimum enclosing ball problem in support vector machine classification. *Applied Sciences*.]

La visualización se hace sobre el espacio de atributos original. Son proyecciones del hiperplano y de los márgenes en dimensiones más altas

$$Pred(x^{(i)}) = \arg \max_{c \in Clases} \hat{P}(Y=c|X=x^{(i)})$$

Support Vector Machine (SVM)

Falto ver:

- Ver cómo permitir instancias “incorrectas” dentro de los márgenes. Soft Margin Classifiers. Sugiero [youtube.com/watch?v=IjSfa708ngs](https://www.youtube.com/watch?v=IjSfa708ngs) y busquen para qué sirve el hiperparámetro C
- Busquen cómo se puede utilizar SVM para **problemas multiclase**.
- Busquen cómo se calcula el **score** de una predicción.
- Ejemplo de uso para anomaly detection: medium.com/@roshmitadey/anomaly-detection-using-support-vectors-2c1b842213ed

¿De todo esto qué tengo que saber?

- Entender que es un Hiperplano de margen máximo
- Entender qué son los vectores de soporte.
- Entender para qué sirve Φ (y saber que no se computa)
- Entender el Kernel Trick

Costo computacional:

De entrenamiento. $O(n^3 * d)$ (pero hay trucos/consideraciones que lo hacen práctico)

Fuente:

scikit-learn.org/stable/modules/svm.html#complexity

2 Answers

Sorted by: Highest score (default)



In order of increasing on-line complexity:

8



1. Linear SVM has prediction complexity $O(d)$ with d the number of input dimensions since it is just a single inner product.
2. NN complexity is related to the architecture, but will surely be above that of linear SVM.
3. Prediction complexity of kernel SVM depends on the choice of kernel and is typically proportional to the number of support vectors. For most kernels, including polynomial and RBF, this is $O(n_{SV} d)$ where n_{SV} is the number of support vectors. [An approximation exists for SVMs with an RBF kernel that reduces the complexity to \$O\(d^2\)\$](#) . For computer vision applications, [additive kernels](#) are often used because they yield very fast prediction speed (independent of the number of SVs).

Modelos Generativos

Enfoques (parte 2)

Vimos hasta ahora: Modelos **DISCRIMINATIVOS** clase con la posterior $Pred(x^{(i)}) = \arg \max_{c \in Clases} P(Y = c | X = x^{(i)})$

Otros métodos surgen de aplicar el teorema de bayes:

$$\begin{aligned} Pred(x^{(i)}) &= \arg \max_{c \in Clases} P(Y = c | X = x^{(i)}) \\ &= \arg \max_{c \in Clases} \frac{P(Y = c)P(X = x^{(i)} | Y = c)}{P(X = x^{(i)})} \\ &= \arg \max_{c \in Clases} P(Y = c) \underbrace{P(X = x^{(i)} | Y = c)}_{\text{Esto es lo que interesa aproximar ahora}} \\ &= \arg \max_{c \in Clases} P(X = x^{(i)}, Y = c) \end{aligned}$$

Esto es lo que
interesa
aproximar ahora

Estos se conocen como modelos **GENERATIVOS**.

ii **Conocer** $P(X = x^{(i)} | Y = c)$ **permite generar muestras!!!**

Para pensar: ¿cómo obtener la probabilidad de una predicción en generativos? $Pred_{\text{proba}}(x^{(i)}, c)$

(algunos) ejemplos de modelos discriminativos:

- **Árboles de decisión** (ya lo vimos)
- **K-vecinos más cercanos** (hoy)
- **Support Vector Machines (SVM)** (hoy)
- **Regresión logística** (prox)
- **Random Forest (y otros ensambles)** (prox)
- **Maximum-entropy Markov models**
- **Conditional random fields**

Teorema de Bayes

$$\underbrace{P(Y = c | X = x^{(i)})}_{\text{posterior}} = \frac{\overbrace{P(X = x^{(i)} | Y = c)}^{\text{likelihood (verosimilitud)}} \cdot \overbrace{P(Y = c)}^{\text{prior}}}{\underbrace{P(X = x^{(i)})}_{\text{marginal likelihood}}}$$

(algunos) ejemplos de modelos generativos:

- **linear discriminant analysis (LDA)** (hoy)
- **gaussian/naive bayes** (hoy)
- **Gaussian mixture model (GMMs)** (prox)
- **Hidden Markov Models (HMMs)**
- **Generative adversarial networks (GANs)**,
- **Autoregressive models** (por ejemplo **GPT-3**)
- **Diffusion models**

modelos generativos

Linear y Quadratic Discriminant Analysis

LDA
QDA



Linear / Quadratic Discriminant Analysis

Método generativo

$$\begin{aligned} \text{Pred}(x^{(i)}) &= \arg \max_{c \in \text{Clases}} P(Y=c|X=x^{(i)}) \\ &= \arg \max_{c \in \text{Clases}} \frac{P(Y=c)P(X=x^{(i)}|Y=c)}{P(X=x^{(i)})} \\ &= \arg \max_{c \in \text{Clases}} P(Y=c) \boxed{P(X=x^{(i)}|Y=c)} \\ &= \arg \max_{c \in \text{Clases}} P(Y=c) \text{pdf}_c(x^{(i)}) \end{aligned}$$

Para X continua

+ sesgo inductivo
+ datos

$$\approx \begin{cases} \arg \max_{c \in \text{Clases}} \hat{P}(Y=c) f_{\text{norm}}(x^{(i)}; \hat{\mu}_c, \hat{\sigma}_c) & \text{para } x^{(i)} \in R & X|Y=c \sim \mathcal{N}(\mu, \sigma) \\ \arg \max_{c \in \text{Clases}} \hat{P}(Y=c) f_{\text{norm}}^d(x^{(i)}; \hat{\mu}_c, \hat{\Sigma}_c) & \text{para } x^{(i)} \in R^d & X|Y=c \sim \mathcal{N}(\mu, \Sigma) \end{cases}$$

$$f_{\text{norm}}(x; \mu, \sigma) = \frac{1}{\sqrt{2\pi}\sigma^2} e^{-\frac{(x-\mu)^2}{2\sigma^2}}$$

$$f_{\text{norm}}^d(\mathbf{x}; \mu, \Sigma) = \frac{1}{\sqrt{(2\pi)^k |\Sigma|}} e^{-\frac{1}{2}(\mathbf{x}-\mu)^T \Sigma^{-1}(\mathbf{x}-\mu)}$$

$$\begin{aligned}
Pred(x^{(i)}) &= \arg \max_{c \in Clases} P(Y=c | X=x^{(i)}) \\
&= \arg \max_{c \in Clases} \frac{P(Y=c)P(X=x^{(i)} | Y=c)}{P(X=x^{(i)})} \\
&= \arg \max_{c \in Clases} P(Y=c)P(X=x^{(i)} | Y=c) \\
&= \arg \max_{c \in Clases} P(Y=c)pdf_c(x^{(i)})
\end{aligned}$$

Linear / Quadratic Discriminant Analysis

Método generativo

(de la diapo anterior, caso general, multidimensional)

$$Pred(x^{(i)}) = \arg \max_{c \in Clases} \hat{P}(Y=c) f_{norm}^d(x^{(i)}; \hat{\mu}_c, \hat{\Sigma}_c) \quad \text{para } x^{(i)} \in R^d$$

- La suposición de estos métodos es que $\mathbf{X} | \mathbf{Y}=\mathbf{c}$ sigue una distribución normal $\mathbf{N}(\mu_c, \Sigma_c)$. Es decir, los puntos en cada clase fueron generados por distribuciones normales distintas.
- Con esta suposición, para modelar la distribución de instancias de cada clase, $P(X=x | Y=c)$, alcanza con **estimar** μ_c , Σ_c , $P(\mathbf{Y}=\mathbf{c})$ usando los datos de entrenamiento.
- Encontrar la clase $\mathbf{c}$ con probabilidad **máxima a posteriori** sale directo (usando las fórmulas de la diapo anterior).

Si la matriz de covarianza (o los desvíos en univariado) se suponen **iguales para toda clase**, este método se llama: **Linear Discriminant Analysis (LDA)**. Si no: **Quadratic Discriminant Analysis (QDA)**

- μ_c y Σ_c pueden ser estimados mediante el método de máxima verosimilitud (con la corrección de Bessel):

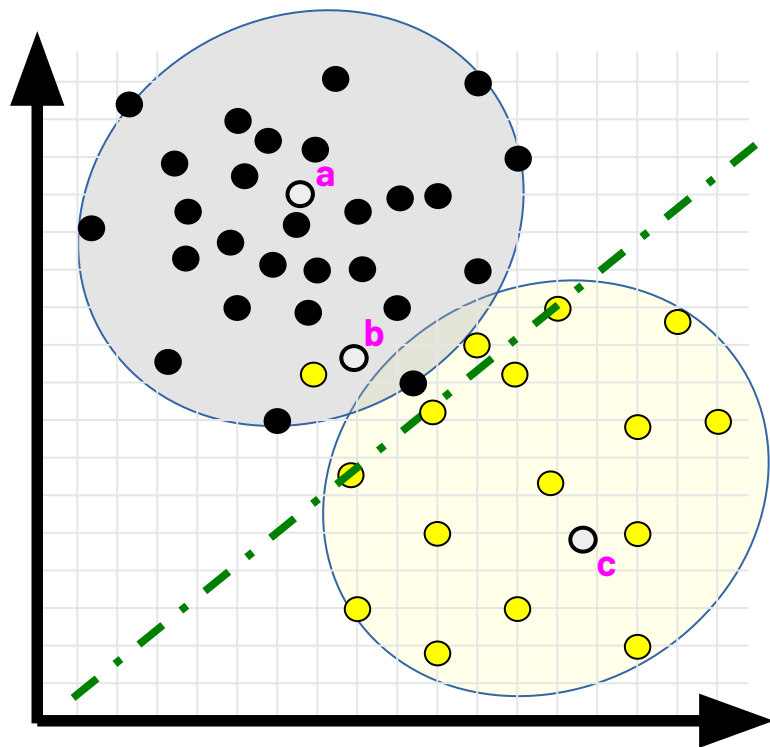
$$\hat{\mu}_c = \frac{1}{n_c} \sum_{x^{(i)} \in \{D|y^{(i)}=c\}} x^{(i)}$$

$$\hat{\Sigma}_c = \frac{1}{n_c - 1} \sum_{x^{(i)} \in \{D|y^{(i)}=c\}} (\mathbf{x}^{(i)} - \hat{\mu}_c)(\mathbf{x}^{(i)} - \hat{\mu}_c)^\top$$

$$\hat{P}(Y=c) = \frac{n_c}{n}$$

Linear / Quadratic Discriminant Analysis

Método generativo



Esquematización (LDA)

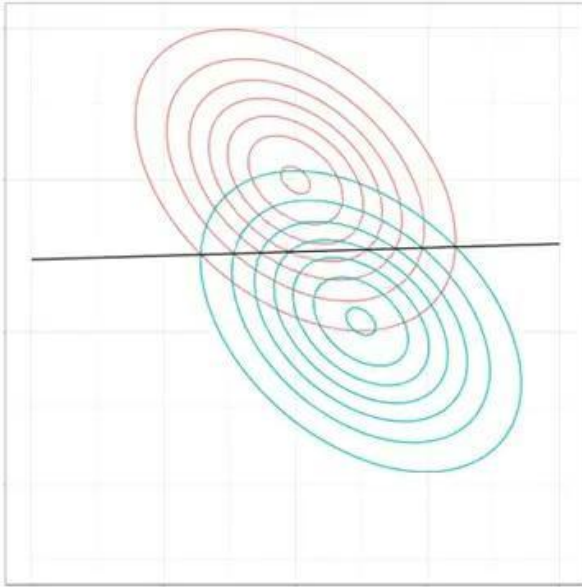
¿De qué clase serán **a**, **b** y **c**?

- 1) Modelar la distribución (normal) de puntos de cada clase;
- 2) Asignar nuevas instancias a la clase que tiene probabilidad máxima a posteriori (MAP).
- 3) La recta verde es la frontera de decisión entre las 2 clases (en este caso, es una recta, es decir LDA).
¿Por qué no está justo al medio?

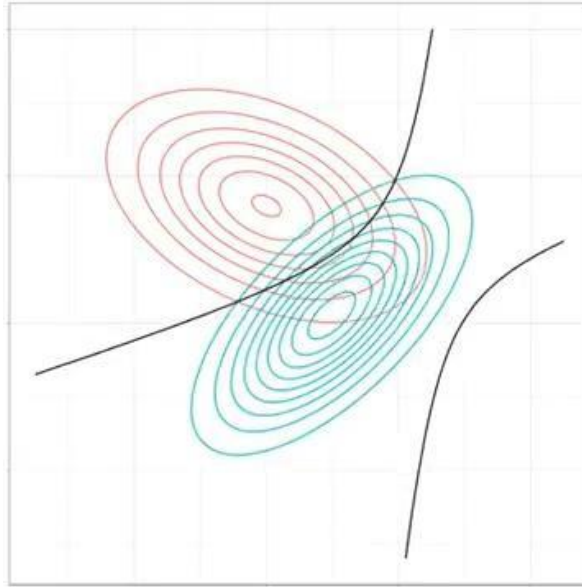
Linear / Quadratic Discriminant Analysis

Método generativo

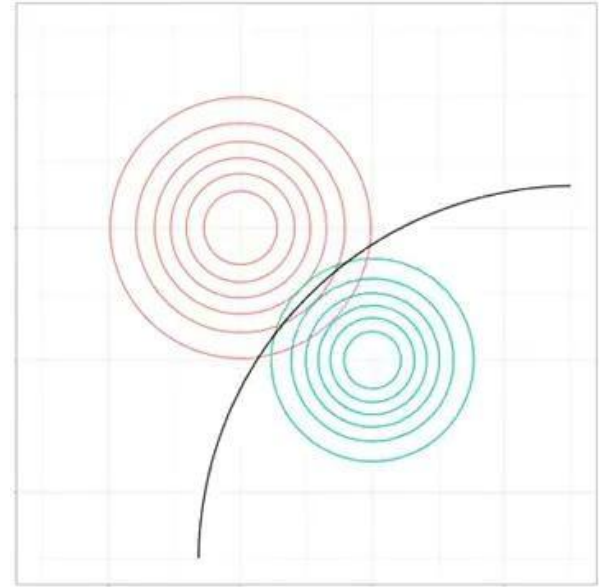
LDA variando la media de una clase



QDA variando la media de una clase



QDA variando la varianza de una clase



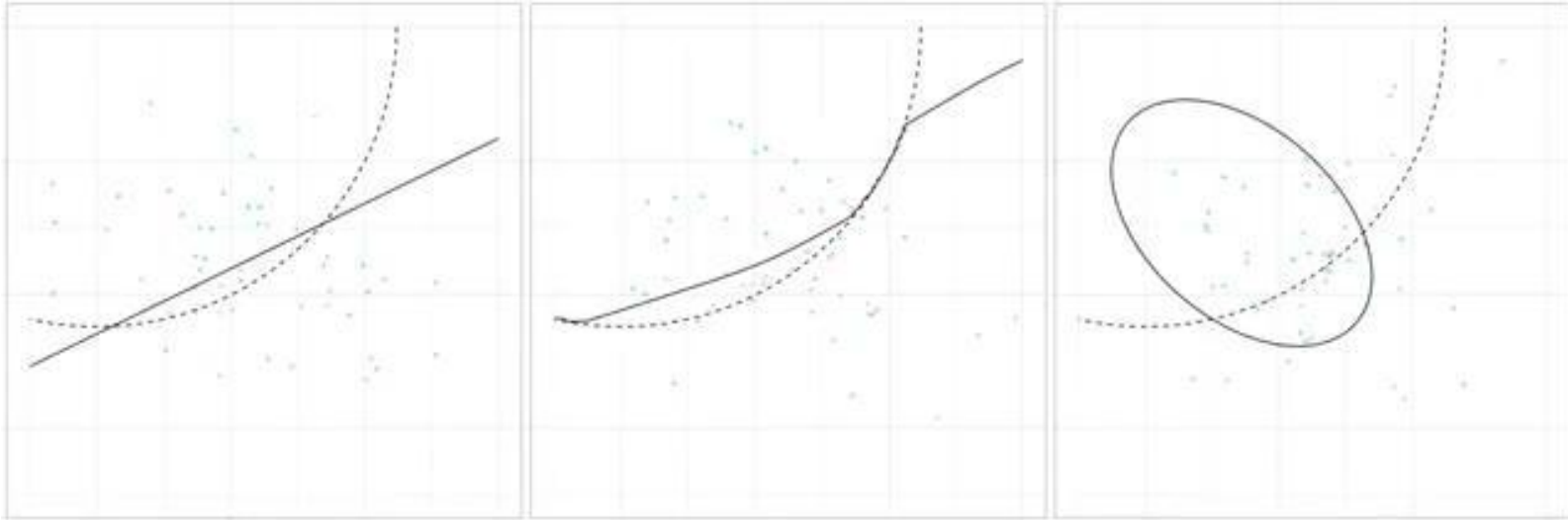
Linear / Quadratic Discriminant Analysis

Método generativo

LDA

QDA (con mat. covarianza diagonal)

QDA general



La línea punteada representa la frontera de decisión óptima (Bayes Optimal Classifier)

Fuente: <https://mathformachines.com/posts/discriminant-analysis/>
VER hilo de twitter (para más animaciones como esta): [Animated-machine-learning-classifiers](#)

modelos generativos

Naive Bayes



Naive Bayes (versión Gaussiana)

Método generativo

$$Pred(x^{(i)}) = \arg \max_{c \in Clases} P(Y=c|X=x^{(i)})$$

$$= \arg \max_{c \in Clases} \frac{P(Y=c)P(X=x^{(i)}|Y=c)}{P(X=x^{(i)})}$$

$$= \arg \max_{c \in Clases} P(Y=c)P(X=x^{(i)}|Y=c)$$

$$= \arg \max_{c \in Clases} P(Y=c)P(X_1 = x_1^{(i)} \wedge \dots \wedge X_p = x_p^{(i)}|Y=c)$$

+ sesgo inductivo
(suposición "Naive")
¿Cuál es?

Para X continua

$$\approx \arg \max_{c \in Clases} P(Y=c) \prod_{j=1}^p P(X_j = x_j^{(i)}|Y=c)$$

$$= \arg \max_{c \in Clases} P(Y=c) \prod_{j=1}^p pdf_c(x_j^{(i)})$$

+ sesgo inductivo
Gaussian Naive Bayes

$$\approx \arg \max_{c \in Clases} \hat{P}(Y=c) \prod_{j=1}^p f_{norm}(x_j^{(i)}; \mu_{\hat{c},j}, \hat{\sigma}_j)$$

Una media por clase, por atributo.

Un desvío por atributo (en general compartido entre clases)

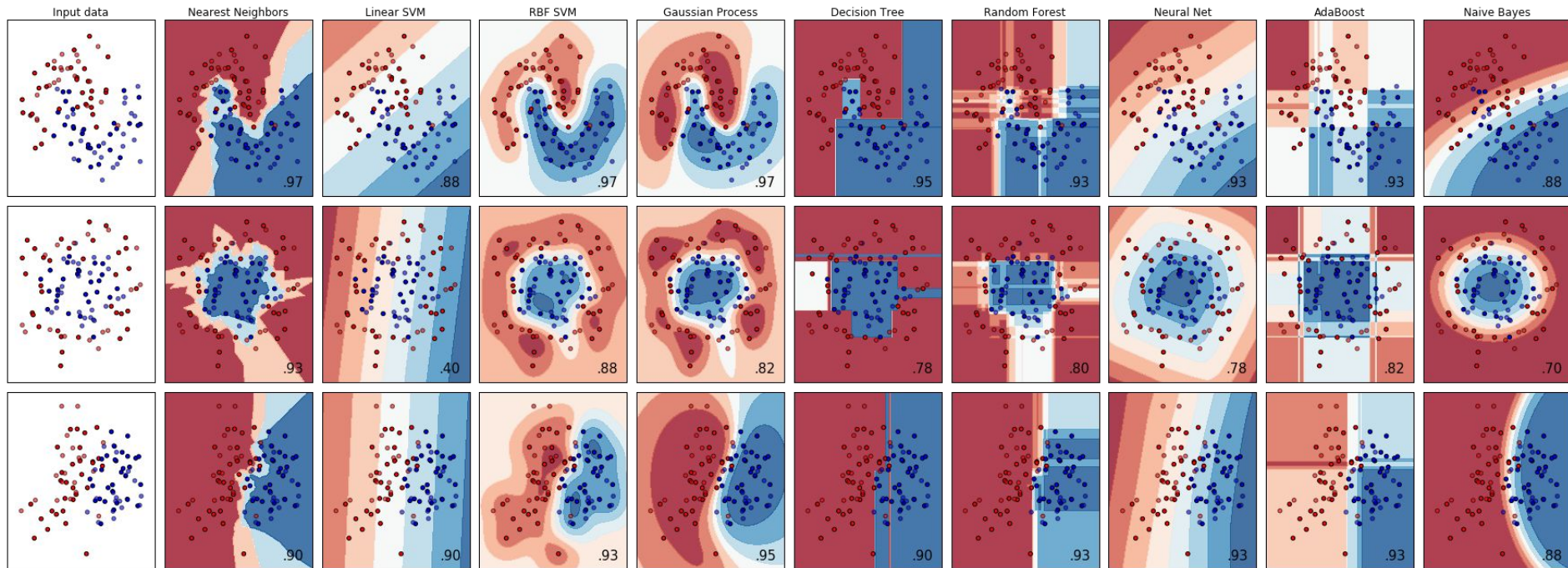
Nota: Modelos paramétricos vs no paramétricos

(Murphy pp.16)

Tiene el modelo una **cantidad fija de parámetros (modelos paramétricos)** o esta cantidad **crece con la cantidad de datos** de entrenamiento **(modelos no paramétricos)**?

¿Cuáles suelen tener “más” sesgo inductivo?

Siempre recordar...



Fuente: https://scikit-learn.org/stable/auto_examples/classification/plot_classifier_comparison.html

Resumen

- **Ensamblés**
- **Bayes Optimal Classifier**
- Modelos **discriminativos** vs **generativos**
- **K Vecinos más Cercanos (KNN)**
- **Support Vector Machines (SVM)**
 - (vimos muy poco, es todo un mundo este tema)
- **Linear Discriminant Analysis (LDA) / Quadratic Discriminant Analysis (QDA)**
- **Naive Bayes Classifier / Gaussian Naive Bayes**

Próximo tema: Métricas

(algunos) ejemplos de modelos discriminativos:

- **Árboles de decisión** (ya lo vimos)
- **K-vecinos más cercanos** (hoy)
- **Support Vector Machines (SVM)** (hoy)
- **Regresión logística** (prox)
- **Random Forest (y otros ensambles)** (hoy)
- **Maximum-entropy Markov models**
- **Conditional random fields**

(algunos) ejemplos de modelos generativos:

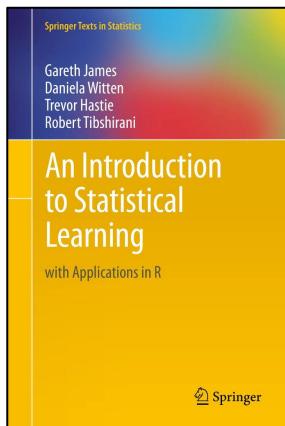
- **linear discriminant analysis (LDA)** (hoy)
- **gaussian/naive bayes** (hoy)
- **Gaussian mixture model (GMMs)** (prox)
- **Hidden Markov Models (HMMs)**
- **Generative adversarial networks (GANs)**,
- **Autoregressive models** (por ejemplo **GPT-3**)
- **Diffusion models**



TAREA

- Leer la [Sec. 4.4 Generative Models for Classification](#) del ISLR.
- [Cap 9 hasta 9.4. Support Vector Machines](#) del ISLR.
- Opcional
 - Recomendado: Leer la [Sec. 4.5 A Comparison of Classification Methods](#) del ISLR.

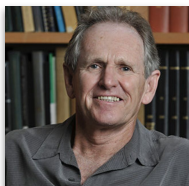
ISLR



Gareth James



Daniela Witten



Trevor Hastie

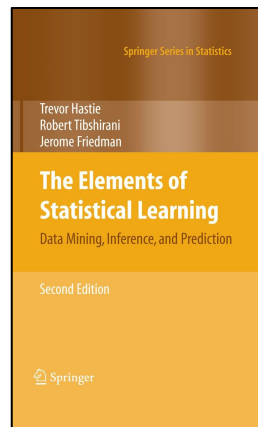


Robert Tibshirani



Jerome Friedman

ESLI



Más sobre SVM

$$Pred(x^{(i)}) = \arg \max_{c \in Clases} \hat{P}(Y=c|X=x^{(i)})$$

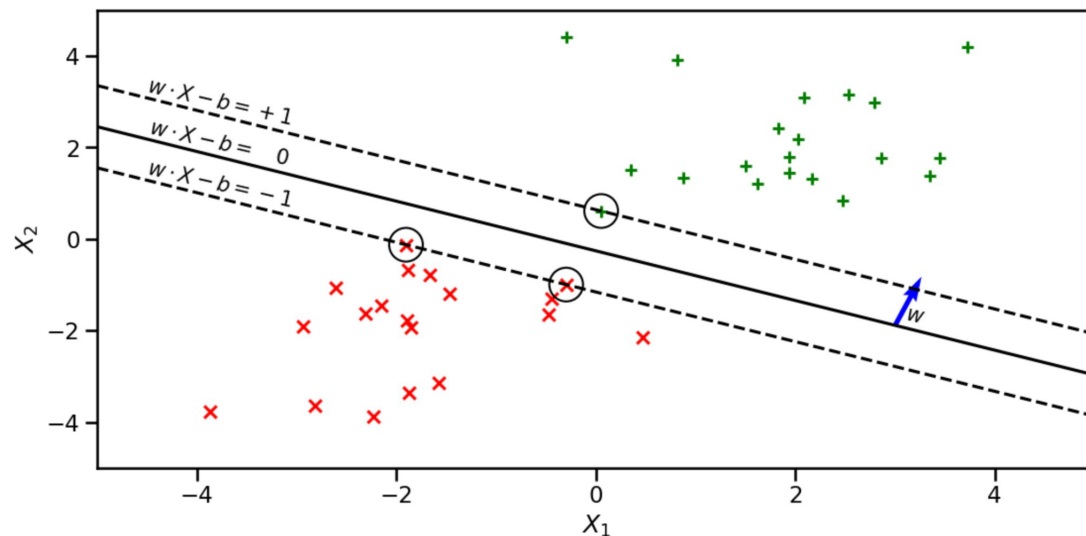
Support Vector Machine (SVM)

Demo: el tamaño del margen es $2 / \|\mathbf{w}\|_2$

Vector unidad en
dirección de $\mathbf{w}$

Sea $\mathbf{p}$ un vector sobre la frontera de decisión. $\mathbf{p}$ tiene que cumplir $\mathbf{w} \cdot \mathbf{p} - b = 0$.

Busquemos $\mathbf{k}$, **cuánto** mover en la dirección de $\mathbf{w}$ para llegar hasta el límite del margen?



$$\begin{aligned} \mathbf{w} \cdot \left(\mathbf{p} + k \cdot \frac{\mathbf{w}}{\|\mathbf{w}\|} \right) - b &= 1 \\ \Leftrightarrow \mathbf{w} \cdot \mathbf{p} + \mathbf{w} \cdot k \cdot \frac{\mathbf{w}}{\|\mathbf{w}\|} - b &= 1 \\ \Leftrightarrow \mathbf{w} \cdot k \cdot \frac{\mathbf{w}}{\|\mathbf{w}\|} &= 1 \\ \Leftrightarrow k \cdot \frac{\|\mathbf{w}\|^2}{\|\mathbf{w}\|} &= 1 \\ \Leftrightarrow k &= \frac{1}{\|\mathbf{w}\|} \end{aligned}$$

El margen es $2\mathbf{k} = 2 / \|\mathbf{w}\|$

Más sobre KNN

$$Pred(x^{(i)}) = \arg \max_{c \in Clases} \hat{P}(Y=c|X=x^{(i)})$$

K vecinos más cercanos (KNN)

Método discriminativo

¿Cómo se implementa? Algoritmo de asignación a la instancia $\mathbf{x}^{(i)}$:

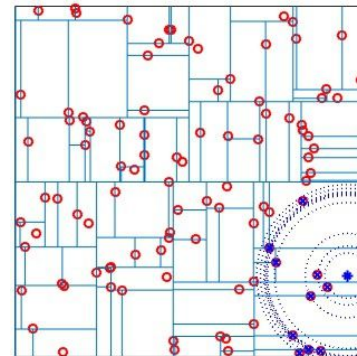
1. Computar la distancia $D(\mathbf{x}^{(i)}, \mathbf{x}^{(j)})$ para todo punto de entrenamiento $\mathbf{x}^{(j)}$.
2. Seleccionar las k instancias más cercanas y sus etiquetas.
3. Devolver la etiqueta más frecuente (o la proporción si es proba).

Costo computacional: $O(n*d + k*n)$ tiempo $O(n)$ memoria
(hay otras opciones dependiendo de decisiones de implementación).

Ver <https://stats.stackexchange.com/questions/219655/k-nn-computational-complexity>

¿Se puede mejorar el costo por query?

Ver estructuras de datos eficientes: "BallTree" y "KDTree".



Fuente: <https://www.vlfeat.org/overview/kdtree.html>